Teach Module 17: JUnit Testing Fundamentals

This README captures the Module 17 teaching session and practice lab. The bootcamp document was used only to identify the Module 17 topic list; the teaching content below is original explanatory material.

Module 17 Topics

Module 17 focuses on JUnit testing fundamentals:

- Why unit tests matter
- JUnit test lifecycle
- Assertions
- Test organization
- Parameterized tests
- Clear test naming and readability
- Reviewing AI/Copilot-generated tests
- Defect lifecycle
- Lab practice

Mockito and mocking are intentionally left out because those belong to the next module.

1. Why Unit Tests Matter

A unit test checks one small piece of code in isolation, usually one method or one behavior.

Example production code:

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

Example unit test:

```
assertEquals(5, calculator.add(2, 3));
```

The goal is not only to prove that code runs. The goal is to prove that a specific behavior remains correct as the code changes.

Good unit tests help developers:

- Catch bugs early
- Refactor safely
- Document expected behavior
- Prevent old bugs from returning
- Build confidence before pushing code

A weak test often tests too much, depends on external systems, or is difficult to understand.

2. JUnit Basics

Modern Java projects commonly use JUnit 5.

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculatorTest {

    @Test
    void addShouldReturnSumOfTwoNumbers() {
        Calculator calculator = new Calculator();

        int result = calculator.add(2, 3);

        assertEquals(5, result);
    }
}
```

Most tests follow the Arrange, Act, Assert pattern:

```
// Arrange
Calculator calculator = new Calculator();

// Act
int result = calculator.add(2, 3);

// Assert
assertEquals(5, result);
```

3. Common Assertions

Assertions tell JUnit what must be true.

```
assertEquals(expected, actual);
assertNotEquals(unexpected, actual);
assertTrue(condition);
assertFalse(condition);
assertNull(value);
assertNotNull(value);
assertThrows(ExceptionType.class, () -> {
    // code expected to throw
});
```

Example with an exception:

```
@Test
void divideShouldThrowExceptionWhenDividingByZero() {
    Calculator calculator = new Calculator();
```

```
    assertThrows(ArithmeticException.class, () -> {
        calculator.divide(10, 0);
    });
}
```

Test both normal behavior and edge cases. Boundary values are especially important.

Example:

```
public boolean isAdult(int age) {
    return age >= 18;
}
```

Useful tests:

```
@Test
void isAdultShouldReturnTrueForAge18() {
    assertTrue(userValidator.isAdult(18));
}

@Test
void isAdultShouldReturnFalseForAge17() {
    assertFalse(userValidator.isAdult(17));
}
```

4. JUnit Lifecycle

JUnit lifecycle annotations let setup or cleanup code run around tests.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculatorTest {

    private Calculator calculator;

    @BeforeEach
    void setUp() {
        calculator = new Calculator();
    }

    @Test
    void addShouldReturnSum() {
        assertEquals(7, calculator.add(3, 4));
    }

    @Test
    void subtractShouldReturnDifference() {
```

```
        assertEquals(2, calculator.subtract(5, 3));
    }
}
```

Common lifecycle annotations:

```
@BeforeEach
@AfterEach
@BeforeAll
@AfterAll
```

Most beginner tests only need `@BeforeEach`.

5. Test Organization

A common Maven project structure:

```
src/main/java
  com/example/Calculator.java

src/test/java
  com/example/CalculatorTest.java
```

Test classes usually mirror production classes:

```
Calculator.java      -> CalculatorTest.java
UserService.java     -> UserServiceTest.java
OrderValidator.java  -> OrderValidatorTest.java
```

Each test should focus on one behavior.

Prefer:

```
@Test
void withdrawShouldReduceBalanceWhenFundsAreAvailable()
```

Avoid:

```
@Test
void testWithdraw()
```

The first name explains the expected behavior.

6. Parameterized Tests

Parameterized tests run the same test logic with multiple inputs.

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;
```

```
import static org.junit.jupiter.api.Assertions.assertTrue;

class UserValidatorTest {

    private final UserValidator validator = new UserValidator();

    @ParameterizedTest
    @CsvSource({
        "18",
        "21",
        "65"
    })
    void isAdultShouldReturnTrueForAdultAges(int age) {
        assertTrue(validator.isAdult(age));
    }
}
```

Example with expected results:

```
@ParameterizedTest
@CsvSource({
    "17, false",
    "18, true",
    "25, true"
})
void isAdultShouldReturnExpectedResult(int age, boolean expected) {
    assertEquals(expected, validator.isAdult(age));
}
```

Use parameterized tests when the logic is the same but the inputs vary.

7. Good Test Names

A good test name answers:

What should happen, and under what condition?

Good examples:

```
calculateDiscountShouldReturnZeroWhenCustomerIsNotEligible()
shouldReturnZeroDiscountForIneligibleCustomer()
validatePasswordShouldFailWhenPasswordIsShorterThanEightCharacters()
```

Weak examples:

```
testDiscount()
testPassword()
test1()
```

8. Reviewing Copilot or AI-Generated Tests

AI can help generate tests, but the developer must review them carefully.

Watch for these problems:

- The test checks the wrong expected value
- The test only verifies the happy path
- The test duplicates the implementation instead of validating behavior
- The test passes even when real code is broken
- The test uses unclear names
- The test unnecessarily depends on time, randomness, files, databases, or network calls

Weak generated test:

```
@Test
void testAdd() {
    Calculator calculator = new Calculator();
    assertEquals(calculator.add(2, 3), calculator.add(2, 3));
}
```

This compares the method to itself, so it proves almost nothing.

Better:

```
@Test
void addShouldReturnSumOfTwoNumbers() {
    Calculator calculator = new Calculator();

    assertEquals(5, calculator.add(2, 3));
}
```

9. Defect Lifecycle

A defect often moves through these stages:

```
Found -> Reproduced -> Diagnosed -> Fixed -> Tested -> Closed
```

JUnit fits into this lifecycle well.

When a bug is found:

1. Write a failing test that reproduces the bug.
2. Fix the production code.
3. Run the test again.
4. Keep the test so the bug does not return.

Example buggy method:

```
public boolean isAdult(int age) {
    return age > 18;
```

```
}
```

Bug: age `18` should count as adult, but the method returns false.

Test:

```
@Test
void isAdultShouldReturnTrueForAge18() {
    assertTrue(validator.isAdult(18));
}
```

Fix:

```
public boolean isAdult(int age) {
    return age >= 18;
}
```

Practice Exercises

Exercise 1: Calculator Tests

Create a `Calculator` class with:

```
add(int a, int b)
subtract(int a, int b)
multiply(int a, int b)
divide(int a, int b)
```

Write JUnit tests for:

- Normal addition, subtraction, multiplication, and division
- Division by zero using `assertThrows`
- Negative numbers
- Zero values

Exercise 2: Age Validator

Create:

```
public boolean isAdult(int age)
```

Rule: age `18` and above is adult.

Test:

- `17` should return false
- `18` should return true
- `19` should return true
- `0` should return false
- Negative age should throw `IllegalArgumentException`

Exercise 3: Password Validator

Create:

```
public boolean isValidPassword(String password)
```

Rules:

- At least 8 characters
- Must contain one uppercase letter
- Must contain one lowercase letter
- Must contain one digit
- Null password should return false or throw an exception, depending on your design

Exercise 4: Parameterized Tests

Create:

```
public boolean isEven(int number)
```

Parameterized test example:

```
@ParameterizedTest
@CsvSource({
    "2, true",
    "3, false",
    "0, true",
    "-4, true",
    "-5, false"
})
void isEvenShouldReturnExpectedResult(int number, boolean expected) {
    assertEquals(expected, numberUtils.isEven(number));
}
```

Exercise 5: String Utility Tests

Create a `StringUtils` class with:

```
reverse(String text)
isPalindrome(String text)
capitalize(String text)
```

Test:

- Normal strings
- Empty strings
- Single-character strings
- Null input
- Case sensitivity for palindrome checks

Exercise 6: Shopping Cart Total

Create a cart that can add item prices and calculate total.

```
cart.addItem(10.00);
cart.addItem(15.50);
double total = cart.calculateTotal();
```

Test:

- Empty cart total is `0`
- One item total
- Multiple item total
- Invalid negative price throws exception

Exercise 7: Defect-Driven Test

Intentionally write a buggy method:

```
public boolean qualifiesForSeniorDiscount(int age) {
    return age > 65;
}
```

Then write a test proving that age `65` should qualify.

```
@Test
void qualifiesForSeniorDiscountShouldReturnTrueForAge65() {
    assertTrue(discountService.qualifiesForSeniorDiscount(65));
}
```

Watch it fail, fix the code to `age >= 65`, then rerun the test.

Exercise 8: Review Bad Tests

Improve this weak test:

```
@Test
void testTotal() {
    Order order = new Order();
    assertNotNull(order.calculateTotal());
}
```

The improved version should check the actual expected total.

Improve this weak test:

```
@Test
void testAdd() {
    assertEquals(calculator.add(2, 3), calculator.add(2, 3));
}
```

Better:

```java
@Test
void addShouldReturnSumOfTwoNumbers() {
    assertEquals(5, calculator.add(2, 3));
}
```

Module 17 Lab: JUnit Testing Fundamentals

Goal

Build a small Java utility project and write JUnit tests for normal behavior, edge cases, exceptions, lifecycle setup, and parameterized tests.

Lab Scenario

You are building validation and calculation utilities for a simple banking-style application.

You will create and test:

```
Calculator
AgeValidator
PasswordValidator
BankAccount
```

Part 1: Project Setup

Create a Maven project with this structure:

```
src/main/java/com/example/
src/test/java/com/example/
```

Add JUnit 5 to `pom.xml`:

```xml
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.10.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Also add:

```xml
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.2.5</version>
```

```
        </plugin>
    </plugins>
</build>
```

Part 2: Create Production Classes

Create `Calculator.java`:

```
package com.example;

public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int divide(int a, int b) {
        return a / b;
    }
}
```

Create `AgeValidator.java`:

```
package com.example;

public class AgeValidator {
    public boolean isAdult(int age) {
        if (age < 0) {
            throw new IllegalArgumentException("Age cannot be negative");
        }
        return age >= 18;
    }
}
```

Create `PasswordValidator.java`:

```
package com.example;

public class PasswordValidator {
    public boolean isValid(String password) {
        if (password == null) {
            return false;
        }

        return password.length() >= 8
                && password.matches(".*[A-Z].*")
                && password.matches(".*[a-z].*")
                && password.matches(".*\\d.*");
    }
}
```

Create `BankAccount.java`:

```
package com.example;

public class BankAccount {
    private double balance;

    public BankAccount(double openingBalance) {
        if (openingBalance < 0) {
            throw new IllegalArgumentException("Opening balance cannot be negative");
        }
        this.balance = openingBalance;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit must be positive");
        }
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal must be positive");
        }

        if (amount > balance) {
            throw new IllegalArgumentException("Insufficient funds");
        }

        balance -= amount;
    }
}
```

Part 3: Write JUnit Tests

Create `CalculatorTest.java`.

Test requirements:

- `add` returns correct sum
- `add` works with negative numbers
- `divide` returns correct result
- `divide` throws `ArithmeticException` when dividing by zero

Example:

```java
@Test
void divideShouldThrowExceptionWhenDividingByZero() {
    Calculator calculator = new Calculator();

    assertThrows(ArithmeticException.class, () -> {
        calculator.divide(10, 0);
    });
}
```

Create `AgeValidatorTest.java`.

Test requirements:

- Age `17` is not adult
- Age `18` is adult
- Age `19` is adult
- Negative age throws `IllegalArgumentException`
- Use a parameterized test for multiple valid adult ages

Create `PasswordValidatorTest.java`.

Test requirements:

- Valid password returns true
- Password shorter than 8 characters returns false
- Password without uppercase returns false
- Password without lowercase returns false
- Password without digit returns false
- Null password returns false
- Use parameterized tests for invalid passwords

Create `BankAccountTest.java`.

Use `@BeforeEach` to create a new account before each test.

Test requirements:

- Opening balance is stored correctly
- Deposit increases balance
- Withdraw decreases balance
- Negative opening balance throws exception
- Zero deposit throws exception
- Negative deposit throws exception
- Withdrawing more than balance throws exception
- Withdrawing zero throws exception

Part 4: Use Clear Test Names

Avoid:

```java
@Test
void testDeposit()
```

Prefer:

```
@Test
void depositShouldIncreaseBalanceWhenAmountIsPositive()
```

Part 5: Run Tests

Run:

```
mvn test
```

All tests should pass.

Part 6: Defect Practice

Temporarily break this line in `AgeValidator`:

```
return age >= 18;
```

Change it to:

```
return age > 18;
```

Run tests again.

Expected result: the test for age `18` should fail.

Then fix it back:

```
return age >= 18;
```

Run tests again and confirm everything passes.

Deliverables

By the end of the lab, you should have:

- 4 production classes
- 4 test classes
- Tests using `@Test`
- Tests using `@BeforeEach`
- Tests using `@ParameterizedTest`
- Tests using `assertEquals`, `assertTrue`, `assertFalse`, and `assertThrows`
- One intentional defect found and fixed by a unit test

Quick Review Questions

1. What is the difference between testing that a value is not null and testing that it is correct?
2. Why is age `18` an important test case for an adult validator?
3. When should you use `assertThrows`?
4. When is a parameterized test better than several separate tests?
5. What makes a test name readable?